

Programming Assignment 1 Hints

REVA

This document details some hints and common questions regarding the disassembler assignment. If you have any followup questions please don't hesitate to reach out via Teams or e-mail!

Please note: This document will be updated as required.

Contents

1 What is the input to our program?	2
2 What is the output of our program?	2
3 How can I create my own examples?	2
4 How do we handle <code>disp8</code> negative displacements?	3
5 test syntax - <code>r/m32, r32</code> vs. <code>r32, r/m32</code>	3
6 Do we need to print <code>BYTE/WORD/DWORD</code> qualifiers?	4
7 Illegal addressing modes for <code>lea</code> and <code>clflush</code>	5
8 How do we handle addressing mode <code>MOD 00</code> with <code>R/M 101</code>?	6
9 What do the opcode abbreviations <code>ib, rb, rw</code>, etc. mean?	8
10 More branching examples	9
11 Differentiating between 16-bit and 32-bit versions of an instruction	11
12 Additional byte stream decoding examples	12

1 What is the input to our program?

Your program must accept one argument using the `-i` parameters. If you are using Python, you may use the `argparse` module. The important point here is that your program must accept the input file through the `-i` parameter.

The supplied input file will be a file that contains the machine code. For example, this could be the output of `nasm`. The input file that you supply **is not** the `.s` files. The `.s` files are the assembly files you can use to assemble with `nasm` that generates the binary files used for your program. In addition, there are a few binary files that have been supplied in the assignment area as well that you may use.

An example run is below. The inputfile is a file that contains the machine code that your disassembler will disassemble.

Example Run:

```
disassembler -i inputfile
```

2 What is the output of our program?

Your program must output to `stdout` (i.e. print to the screen). You may print to a file as well, but the screen output is what will be graded.

Example Output:

```
00000000: 55          push ebp
00000001: 89E5        mov ebp,esp
00000003: 52          push edx
00000004: 51          push ecx
00000005: 39D1        cmp ecx,edx
00000007: 740F        jz offset_00000018h
00000009: B844434241  mov eax,0x41424344
0000000E: 8B5508      mov edx,[ebp+0x00000008]
00000011: 8B4D0C      mov ecx,[ebp+0x0000000c]
00000014: 01D1        add ecx,edx
00000016: 89C8        mov eax,ecx
offset_00000018h:
00000018: 5A          pop edx
00000019: 59          pop ecx
0000001A: 5D          pop ebp
0000001B: C20800      retn 0x0008
```

3 How can I create my own examples?

The simplest way to do this is utilizing `nasm`. One thing to remember is that often times assemblers such as `nasm` will only assemble an assembly instruction using a specific opcode. You can utilize a `nasm` directive to force output of a specific sequence of bytes.

For example, with the instruction `pop eax`, the assembler will want to minimize size and utilize the `0x58 + rd` opcode. Some cases, we may be able to override the default `nasm` behavior. In cases where we can't, we can utilize the `nasm` directive `db` to declare bytes whereby directing `nasm` to emit the byte or sequence of bytes.

Example NASM:

```
[BITS 32]

pop eax
db 0x8f
db 0xc0
```

In the above listing, we see that we utilize the `nasm` directive `db` to emit two bytes: `0x8F` and `0xC0`. The opcode `8F /0` represents the assembly `pop r/m32`. Since `eax` is a 32-bit register, we can use this instruction in direct register mode, which is addressing mode `11b` (or `3`).

Example disassembly:

```
00000000: 58          pop eax
00000001: 8FC0       pop eax
```

We've successfully created two distinct machine code that results in the same assembly.

4 How do we handle `disp8` negative displacements?

`disp8` are also signed values. When they are displayed, please ensure they are being displayed properly. You can do this in two ways. Below is an example showing both valid ways that will receive full credit. Below that is an incorrect display that will not get credit. Assume the follow byte sequence is encountered by your disassembler.

```
01 47 FC
```

Correct Representations:

```
add [ edi - 0x04 ], eax
```

or

```
add [ edi + 0xFFFFFFFFFC ], eax
```

Incorrect Representation:

```
add [ edi + 0xFC ], eax
```

5 test syntax - `r/m32, r32` vs. `r32, r/m32`

The Intel SDM shows that `test` syntax should be `test r/m32, r32`. Why does the example have a `test r32, r/m32`?

Either syntax is fine. The assembler is translating the syntax for us. Thus, while the Intel Manual and correct syntax is `test r/m32, r32`, the assembler is converting

`test r32, r/m32` for us. In your disassembler, either output would be acceptable. The reason the assemblers are generally lenient with this syntax in the `test` (and some other similar instructions) is because there is no **write** operation happening, thus the order does not matter.

To be clear, if you saw `85 31` you could output either of the following:

```
test [ecx], esi
```

or

```
test esi, [ecx]
```

6 Do we need to print BYTE/WORD/DWORD qualifiers?

No, your disassembler does not need to output the appropriate qualifier (e.g. `byte`, `word`, `dword [ptr]`). Some assemblers require these qualifiers to provide specifics as to what size value is being represented.

In most cases, when you are referencing a memory access, the assembler (especially `nasm` and `masm`) will require you to provide the qualifier. Instead of:

```
add [eax], 0x11223344
```

it is expecting

```
add dword [eax], 0x11223344
```

7 Illegal addressing modes for `lea` and `clflush`

For `lea` and `clflush`, the addressing mode 11 (direct register) is illegal. However, the other 3 modes are just fine.

Examples:

```
(MOD 00)      8D 31      lea esi, [ ecx ]
(MOD 00/RM 101) 8D 35 44 43 42 41 lea esi, [ 0x41424344 ]
(MOD 01)      8D 71 04      lea esi, [ ecx + 0x04 ]
(MOD 10)      8D B1 44 43 42 41 lea esi, [ ecx + 0x41424344 ]
```

Legal but uses SIB

```
(MOD 00)      8D 34 31      lea esi, [ ecx + esi ]
(MOD 01)      8D 74 31 04      lea esi, [ ecx + esi + 0x04 ]
(MOD 10)      8D B4 31 44 43 42 41 lea esi, [ ecx + esi + 0x41424344 ]
```

Illegal Example:

```
lea esi, ecx ; be sure to treat this as 'invalid/unknown opcode'
```

The `clflush` instruction is a Cache Line Flush. The address that is specified is evicted from every level of the cache hierarchy. If that particular address has been modified, the changes will be written back to memory. Like the `lea` instruction, the brackets do not indicate a memory reference, but rather an address calculation. Thus, in the examples below, the calculated address is the address to be evicted. We will be learning more about this instruction, as it pertains to understanding some of the recent processor attacks (specifically, a timing-based side-channel attack to leak sensitive data).

```
00000000  0F AE 38      clflush [eax]
00000003  0F AE 78 08      clflush [eax+0x8]
00000007  0F AE B8 08 00 00 00 clflush [eax+0x8]
0000000E  0F AE 7C BE 08      clflush [esi+edi*4+0x8] ; requires SIB
00000013  0F AE BC BE 08 00 00 00 clflush [esi+edi*4+0x8] ; requires SIB
0000001B  0F AE 3C BE      clflush [esi+edi*4] ; requires SIB
0000001F  0F AE 3D 44 33 22 11 clflush [0x11223344]
```

Illegal Example:

```
clflush esi ; be sure to treat this as 'invalid/unknown opcode'
```

8 How do we handle addressing mode MOD 00 with R/M 101?

I will use the Example 4 found on page 14 of the Intel Instruction Tutorial as the example: The `r/m32` operand determines which addressing mode we are in. Thus, if the `r/m32` operand is a `[disp32]` only (whether it is a source operand or destination operand), this is encoded using addressing mode 00. In this addressing mode, if the `r/m` field is set to 101, this will indicate a `disp32` is the memory reference, **not** `ebp`.

```
mov [ 0x11223344 ], ebx
mov [ disp32 ], r32
```

1. We look up the `mov` instruction in the manual and see that it is on page Vol 2B 4-35.
2. We look for the syntax that matches ours (i.e. the left operand is a memory access while the right operand is direct register). We see that the `0x89` opcode has the syntax `r/m32, r32` whereas the `0x8b` has the syntax `r32, r/m32`. The `0x8b` syntax will not work since the first operand is a direct register access (`r/m32` is the Intel syntax for register or memory access while `r32` is direct register access). Thus, we must choose the `0x89` opcode.
3. Looking at the `Op/En` we see that the opcode `0x89` has an `Op/En` of **MR** (**MR** indicates the first operand should go in the `r/m` location). Looking at the Instruction Operand Encoding table on the next page of the Intel SDM, we see the **ModR/M** byte is required.
4. Using our diagram from class, we know that the top 2 bits of the **ModR/M** byte are used to determine if this is a memory offset or direct register access. Since `[disp32]` is an indirect memory access only, we set the top 2 bits to be 00 and we set `r/m` to 101 (See Figure 4 for clarification). The next 3 bits indicate the source register (in this case `ebx`). Looking at the table from class, `ebx` is 011. Thus, the **ModR/M** byte is 00011101 or `0x1D`.
5. Since there is a `disp32`, we must encode this next. This is simply `0x44 0x33 0x22 0x11`.
6. The final instruction is `0x89 0x1D 0x44 0x33 0x22 0x11`.

Now, if we wanted to disassemble the byte stream:

`0x89 0x1D 0x44 0x33 0x22 0x11`, we could do the following:

If we reverse the steps (focusing on the **ModR/M** byte), we can see that the **ModR/M** byte is `0x1D`.

- ```
0x1D = 00 011 101
mode = 00
reg = 011
r/m = 101
```

$$r/m = 101$$

- ### Table 2-2. 32-Bit Addressing Forms with the ModR/M Byte

|                                                                                                                                                                              |                                            |                                                      |                                              |                                              |                                              |                                              |                                              |                                              |                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|
| r8(r/r)<br>r16(r/r)<br>r32(r/r)<br>mm(r/r)<br>xmm(r/r)<br>(In decimal) /digit (Opcode)<br>(In binary) REG =                                                                  | AL<br>AX<br>EAX<br>MM0<br>XMM0<br>0<br>000 | CL<br>CX<br>ECX<br>MM1<br>XMM1<br>1<br>001           | DL<br>DX<br>EDX<br>MM2<br>XMM2<br>2<br>010   | BL<br>BX<br>EBX<br>MM3<br>XMM3<br>3<br>011   | AH<br>SP<br>ESP<br>MM4<br>XMM4<br>4<br>100   | CH<br>BP<br>EBP<br>MM5<br>XMM5<br>5<br>101   | DH<br>SI<br>ESI<br>MM6<br>XMM6<br>6<br>110   | BH<br>DI<br>EDI<br>MM7<br>XMM7<br>7<br>111   |                                              |
| Effective Address                                                                                                                                                            | Mod                                        | R/M                                                  | Value of ModR/M Byte (in Hexadecimal)        |                                              |                                              |                                              |                                              |                                              |                                              |
| [EAX]<br>[ECX]<br>[EDX]<br>[EBX]<br>[ ]-1<br>disp32 <sup>2</sup><br>[ESI]<br>[EDI]                                                                                           | 00                                         | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | 00<br>01<br>02<br>03<br>04<br>05<br>06<br>07 | 08<br>09<br>0A<br>0B<br>0C<br>0D<br>0E<br>0F | 10<br>11<br>12<br>13<br>14<br>15<br>16<br>17 | 18<br>19<br>1A<br>1B<br>1C<br>1D<br>1E<br>1F | 20<br>21<br>22<br>23<br>24<br>25<br>26<br>27 | 28<br>29<br>2A<br>2B<br>2C<br>2D<br>2E<br>2F | 30<br>31<br>32<br>33<br>34<br>35<br>36<br>37 |
| [EAX]+disp8 <sup>3</sup><br>[ECX]+disp8<br>[EDX]+disp8<br>[EBX]+disp8<br>[ ]-+disp8<br>[EBP]+disp8<br>[ESI]+disp8<br>[EDI]+disp8                                             | 01                                         | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | 40<br>41<br>42<br>43<br>44<br>45<br>46<br>47 | 48<br>49<br>4A<br>4B<br>4C<br>4D<br>4E<br>4F | 50<br>51<br>52<br>53<br>54<br>55<br>56<br>57 | 58<br>59<br>5A<br>5B<br>5C<br>5D<br>5E<br>5F | 60<br>61<br>62<br>63<br>64<br>65<br>66<br>67 | 68<br>69<br>6A<br>6B<br>6C<br>6D<br>6E<br>6F | 70<br>71<br>72<br>73<br>74<br>75<br>76<br>77 |
| [EAX]+disp32<br>[ECX]+disp32<br>[EDX]+disp32<br>[EBX]+disp32<br>[ ]-+disp32<br>[EBP]+disp32<br>[ESI]+disp32<br>[EDI]+disp32                                                  | 10                                         | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | 80<br>81<br>82<br>83<br>84<br>85<br>86<br>87 | 88<br>89<br>8A<br>8B<br>8C<br>8D<br>8E<br>8F | 90<br>91<br>92<br>93<br>94<br>95<br>96<br>97 | 98<br>99<br>9A<br>9B<br>9C<br>9D<br>9E<br>9F | A0<br>A1<br>A2<br>A3<br>A4<br>A5<br>A6<br>A7 | A8<br>A9<br>AA<br>AB<br>AC<br>AD<br>AE<br>AF | B0<br>B1<br>B2<br>B3<br>B4<br>B5<br>B6<br>B7 |
| EAX/AX/AL/MM0/XMM0<br>ECX/CX/CL/MM1/XMM1<br>EDX/DX/DL/MM2/XMM2<br>EBX/BX/BL/MM3/XMM3<br>ESP/SP/AH/MM4/XMM4<br>EBP/BP/CH/MM5/XMM5<br>ESI/SI/DH/MM6/XMM6<br>EDI/DI/BH/MM7/XMM7 | 11                                         | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | C0<br>C1<br>C2<br>C3<br>C4<br>C5<br>C6<br>C7 | C8<br>C9<br>CA<br>CB<br>CC<br>CD<br>CE<br>CF | D0<br>D1<br>D2<br>D3<br>D4<br>D5<br>D6<br>D7 | D8<br>D9<br>DA<br>DB<br>DC<br>DD<br>DE<br>DF | E0<br>E1<br>E2<br>E3<br>E4<br>E5<br>E6<br>E7 | E8<br>E9<br>EA<br>EB<br>EC<br>ED<br>EE<br>EF | F0<br>F1<br>F2<br>F3<br>F4<br>F5<br>F6<br>F7 |

2. The disp32 nomenclature denotes a 32-bit displacement that follows the ModR/M byte (or the SIB byte if one is present) and that is added to the index.

- 7

## 9 What do the opcode abbreviations **ib**, **rb**, **rw**, etc. mean?

You can find this in the Intel manual, specifically in 3.1.1.1 and you may also find 3.1.1.3 useful as well.

In each case the “b” means byte (8-bit value), “w” means word (16-bit value), “d” means dword (32-bit value) and “o” refers to qword (64-bit value). Effectively the “c” means a code offset (this is encoded the same as an immediate (i.e. little-endian) and represents the code offset for a branch instruction), the “i” means immediate (e.g. `mov eax, 0x11223344`; `0x11223344` is an immediate [specifically `id`], and the `+rb/+rw/+rd/+ro` indicate that you take the opcode (e.g. `0x50` for a `push`) and you add to it the register value (e.g. `ebp = 101` or `5`; `0x50 + 5 = 0x55`).

The explanation below is directly from the Intel Manual as well for your reference:

NP — Indicates the use of 66/F2/F3 prefixes (beyond those already part of the instructions opcode) are not allowed with the instruction. Such use will either cause an invalid-opcode exception (#UD) or result in the encoding for a different instruction.

/digit — A digit between 0 and 7 indicates that the ModR/M byte of the instruction uses only the r/m (register or memory) operand. The reg field contains the digit that provides an extension to the instruction’s opcode.

/r — Indicates that the ModR/M byte of the instruction contains a register operand and an r/m operand.

cb, cw, cd, cp, co, ct — A 1-byte (cb), 2-byte (cw), 4-byte (cd), 6-byte (cp), 8-byte (co) or 10-byte (ct) value following the opcode. This value is used to specify a code offset and possibly a new value for the code segment register.

ib, iw, id, io — A 1-byte (ib), 2-byte (iw), 4-byte (id) or 8-byte (io) immediate operand to the instruction that follows the opcode, ModR/M bytes or scale-indexing bytes. The opcode determines if the operand is a signed value. All words, doublewords and quadwords are given with the low-order byte first.

+rb, +rw, +rd, +ro — Indicated the lower 3 bits of the opcode byte is used to encode the register operand without a modR/M byte. The instruction lists the corresponding hexadecimal value of the opcode byte with low 3 bits as 000b. In non-64-bit mode, a register code, from 0 through 7, is added to the hexadecimal value of the opcode byte. In 64-bit mode, indicates the four bit field of REX.b and opcode[2:0] field encodes the register operand of the instruction. “+ro” is applicable only in 64-bit mode. See Table 3-1 for the codes.



## 10 More branching examples

Consider the following two examples (first is 32-bit second is 8-bit).

### FIRST EXAMPLE (`re132`)

**Note:** The same reasoning below is true for the `call re132` instruction as well. We will focus on the `jmp` at offset `0x01010008`. To calculate the location we perform the following steps:

1. Recognize the opcode (`0xE9`) is a `jmp re132`.
2. Recognize the instruction (`jmp`) is located at address `0x01010008`.
3. Obtain the `re132` displacement by reading in the next 4 bytes and treat them as a 32-bit value: `0xFFFFFFFF3`.
4. Add the current instruction address (`0x01010008`) to the size of the instruction (5) and finally to the 32-bit displacement (`0xFFFFFFFF3`).  
$$0x01010008 + 0x00000005 + 0xFFFFFFFF3 = 0x101010000$$
5. We recognize that this new value (`0x101010000`) is actually a 33-bit value. We have to take the low 32-bits only and ignore the overflow. Thus the final branch target is: `0x01010000`

```
offset_01010000:
01010000: C3 retn
01010001: 90 nop
01010002: 90 nop
01010003: 90 nop
01010004: 90 nop
01010005: 90 nop
01010006: 90 nop
01010007: 90 nop
01010008: E9 F3 FF FF FF jmp offset_01010000
```

### SECOND EXAMPLE (`re18`)

Again, we will focus on the `jmp` at offset `0x01010008`. To calculate the location we perform the following steps:

1. Recognize the opcode (`0xEB`) is a `jmp re18`.
2. Recognize the instruction (`jmp`) is located at address `0x01010008`.
3. Obtain the `re18` displacement by reading in the next byte (`0xF6`) and treat it as a sign-extended 32-bit value: `0xFFFFFFFF6`. How this is done is language dependent. In Java, everything is a signed value, so you can use it as is. In C, you need to make sure this value is read as a signed value (ensure you don't have the `unsigned` qualifier by it).

Example:

```
// assumes currentInstruction is 0x01010008
char offset = currentInstruction[1];

// this next line sign extends 0xF6 to 0xFFFFFFFF6.
int disp32 = (int)offset;
```

4. Add the current instruction address (0x01010008) to the size of the instruction (2) and finally to the 32-bit displacement (0xFFFFFFFF6).

$0x01010008 + 0x00000002 + 0xFFFFFFFF6 = 0x101010000$

5. We recognize that this new value (0x101010000) is actually a 33-bit value. We have to take the low 32 bits only and ignore the overflow. Thus the final branch target is: 0x01010000.

```
offset_01010000:
01010000: C3 retn
01010001: 90 nop
01010002: 90 nop
01010003: 90 nop
01010004: 90 nop
01010005: 90 nop
01010006: 90 nop
01010007: 90 nop
01010008: EB F6 jmp offset_01010000
```

## 11 Differentiating between 16-bit and 32-bit versions of an instruction

In the tables for many of the instructions, the same opcode is listed with both 16-bit and 32-bit versions. For example, `add` has the following entries (among others):

|          |                  |
|----------|------------------|
| 05 iw    | ADD ax, imm16    |
| 05 id    | ADD eax, imm32   |
| 81 /0 iw | ADD r/m16, imm16 |
| 81 /0 id | ADD r/m32, imm32 |
| 83 /0 ib | ADD r/m16, imm8  |
| 83 /0 ib | ADD r/m32, imm8  |

How do you differentiate between these cases? In particular, for the ones that have a 2-byte vs. 4-byte immediate argument following it, how does the processor determine which one it is? Is it just the case that all 32-bit (or 64-bit) processors use the 32-bit/DWORD versions, and the manual is just showing the 16-bit versions for backwards reference?

### Answer:

For the assignment, you won't need to differentiate.

In practice, the processor can differentiate between the 16-bit and 32-bit version by keeping track of the mode the processor is running in. For example, if the processor is running in 32-bit mode, and it runs across a 05 or 81 or 83, it will treat the values as `imm32` and `r/m32`. If the processor is running in 16-bit mode, it will treat them as `imm16` and `r/m16`.

To switch between the processing modes (and this can extend to 64-bit as well), we can use an **Operand Override Prefix** (0x66). Thus, when the processor comes across a 05 or 81 or 83 **preceded by an operand prefix** (0x66) it will switch modes for the instruction access. It will treat the operands as the other interpretation (that is if the processor is running in 32-bit mode and it executes 66 05 22 11, it will be `mov ax, 0x1122`).

In `nasm` syntax, the assembler will handle this for you:

```
add eax, 0x11223344 ; will get encoded as 05 44 33 22 11
add ax, 0x1122 ; will get encoded as 66 05 22 11
```

Similar, but not related to the operand override prefix, we can also have the assembler treat values as bytes, words or dwords, using these keywords as qualifiers.

```
push byte 0x01
push strict dword 0x01
```

If we assemble the above with `nasm` and see the encoding is:

```
00000000 6A01 push byte +0x1
00000002 6801000000 push dword 0x1
```

## 12 Additional byte stream decoding examples

How can we decode the following?

```
814104443322 ==> add dword [byte ecx + 4], 0x11223344
```

```
81810400000044332211 ==> add dword [dword ecx + 4], 0x11223344
```

The examples are very similar, but I will break them down separately.

**Note:** The `byte ecx + 4` is a hint to the assembler to assemble the **displacement as a 1-byte value**. Likewise, the `dword ecx + 4` is a hint to the assembler to assemble the **displacement as a 4-byte value**. The first `dword` in each case is there to simply let the assembler know this memory access is a 32-bit memory access. You don't need to show the `dword` or `byte` qualifiers in your disassembly. `ndisasm` is using that to show you the breakdown. Some assemblers may require you to use these qualifying words, however, for the assignment you are not required to.

### FIRST EXAMPLE

1. You need to inspect the first byte `0x81`. Thus, this can be any of the following instructions. Therefore, we must inspect the **ModR/M** byte (specifically the **reg** field) to determine which instructions this is.

```
add r/32, imm32 0x81 /0
or r/32, imm32 0x81 /1
adc r/32, imm32 0x81 /2
sbb r/32, imm32 0x81 /3
and r/32, imm32 0x81 /4
sub r/32, imm32 0x81 /5
xor r/32, imm32 0x81 /6
cmp r/32, imm32 0x81 /7
```

2. Let's look at the **ModR/M** byte (`0x41`).

- **MOD** = 01 (indirect memory access with 1-byte displacement)
- **REG** = 000 (0 indicates the extension which means this is an `add` instruction)
- **R/M** = 001 (1 indicates `r/m32` is the `ecx` register)

3. Since the addressing mode is 01, we know that a 1-byte displacement follows. In this case, `0x04`.
4. Since the 81 opcode is `add r/m32, imm32`, we know that the next 4 bytes are the immediate: `0x11223344`.
5. The final decoding would be: `add [ ecx + 0x04 ], 0x11223344`.

## SECOND EXAMPLE

1. You need to inspect the first byte **0x81**. Thus, this can be any of the following instructions. Therefore, we must inspect the **ModR/M** byte (specifically the **reg** field) to determine which instructions this is.

```
add r/32, imm32 0x81 /0
or r/32, imm32 0x81 /1
adc r/32, imm32 0x81 /2
sbb r/32, imm32 0x81 /3
and r/32, imm32 0x81 /4
sub r/32, imm32 0x81 /5
xor r/32, imm32 0x81 /6
cmp r/32, imm32 0x81 /7
```

2. Let's look at the **ModR/M** byte (**0x81**).

- **MOD** = 10 (indirect memory access with 4-byte displacement)
- **REG** = 000 (0 indicates the extension which means this is an **add** instruction)
- **R/M** = 001 (1 indicates r/m32 is the ECX register)

3. Since the addressing mode is 10, we know that a 4-byte displacement follows. In this case, **0x00000004**.
4. Since the 81 opcode is **add r/m32, imm32**, we know that the next 4 bytes are the immediate: **0x11223344**.
5. The final decoding would be: **add [ ecx + 0x04 ], 0x11223344**.